

A decomposition-dependent adjoint in DAFoam v5

and two adjacent defects of the MACH-Aero toolchain: the IDWarp rotation guard, and the ILU conditioning wall

Certonomous CFD Lab

2026-08-05

Abstract

On a 2,777-cell OpenFOAM case run under `dafoam/opt-packages:latest` (DAFoam 5.0.0, OpenFOAM v2506), varying *only* the mesh decomposition moves the analytic $dC_D/d(\text{shape})$ gradient from 0.00054% agreement with its own finite difference (`simple` 4x1x1) to 8.95% disagreement (`scotch`, the shipped default), with the converged primal invariant to five significant figures and the adjoint right-hand side invariant to 4.8×10^{-6} . The linear solve is not at fault: it converges cleanly (`PetscConvergedReason: 2`, true residual 1.7×10^{-7} in the unpreconditioned norm) and is unmoved by a tolerance tightened four decades. The discriminating measurement is a cross-residual: the converged `scotch` adjoint vector, permuted to serial ordering by integer `decomposePar` addressing and evaluated under the serial reverse-AD operator, leaves a residual of $329 \times \|b\|$ against that operator's own 1.14×10^{-4} floor — concentrated in the x -momentum rows of partition-interface cells. The parallel matrix-free transposed-Jacobian product is not the transpose Jacobian of the residual it claims to differentiate. We report the phenomenon, the discriminators that convicted the operator and exonerated every other candidate, the localization, why no published verification could have caught it, and two adjacent findings from the same forensic arc: an IDWarp degenerate-branch guard that differentiates to a hard zero at exactly the state every gradient evaluation uses (root-caused, patched in four lines, and answering a five-year-old open upstream issue), and an ASM sub-block ILU factorization whose failure is broken by the PETSc developers' own sanctioned escalation to complete LU. **Nothing in this document has been filed upstream.**

Status. No issue has been opened, no maintainer contacted, nothing posted. This document is the extended, citable companion to two prepared-but-unfiled reports, `demo-output/website/dafoam/UPSTREAM_BUG_REPORT_decomposition_adjoint.md` and `demo-output/website/dafoam/UPSTREAM_BUG_REPORT_mesh_warpDeriv.md`. It does not replace them. Where our own record was found defective by our own verification, the defect is printed here rather than repaired silently.

Contents

1	Summary of findings	2
2	The decomposition-dependent adjoint	3
2.1	The phenomenon	3
2.2	The discriminators	4
2.3	The objection this measurement must survive	6
2.4	Sign-convention provenance, stated against ourselves	6
2.5	Localization	7
2.6	The mechanism, named — and what is not claimed	7
2.7	A usability finding recorded on the way	8
2.8	How far the defect reaches	8

3	Why it was never caught	9
3.1	The papers' own protocols	9
3.2	The community record	11
4	The IDWarp rotation guard	12
4.1	The line	12
4.2	Narrowing the claim, against ourselves	13
4.3	Why upstream's CI cannot see it	14
4.4	The upstream issue this answers	14
4.5	The patch, and what it proves	15
4.6	The second regime, and the fix that covers both	16
4.7	Workaround, with its price named	16
5	The conditioning wall, and the sub-LU remedy	17
5.1	The wall	17
5.2	The offline reproducer, which made every candidate a ten-second experiment . . .	17
5.3	Runtime options: reachable, measured, and empty	17
5.4	The one-word change, offline and then in-solver	18
5.5	The gradient, and its verification	18
5.6	Where the remedy comes from, and why it is not exotic	19
5.7	What the change does <i>not</i> fix	19
6	Methods: adversarial verification as practice	19
6.1	What a supervisor sweep is	20
6.2	What the sweeps actually found	20
6.3	Pre-registration	20
6.4	Cost, stated including the overruns	22
6.5	The lessons this arc produced	22
A	Reproduction appendix	23
A.1	Environments	23
A.2	Switches and stamps	23
A.3	Varying the decomposition without falling into the trap	24
A.4	The cross-residual protocol	24
A.5	Reproducing the IDWarp defect in isolation	25
A.6	Rebuilding the sub-LU image	25
A.7	Artifact index	25
A.8	What is still open	26

1 Summary of findings

Three findings, from three weeks of forensic work on the MACH-Aero/DAFoam toolchain. Two are defects in shipped code; the third is a wall in a shipped default with a remedy that is one option away.

What links them. All three are invisible to the checks that ship with the tools. D2's discarded term sums to *exactly* zero over the surface, and the sum is the only quantity upstream's test asserts on. D1 produces a gradient that is self-consistent under the decomposition it was measured on, and no published verification varies the decomposition. D3 announces itself loudly — but the documented remedy ladder does not contain the remedy. Each was found by measuring the thing the instrument was not looking at.

Finding	Where	Headline measurement	Status
D1. Decomposition-dependent adjoint	DAFoam v5, matrix-free (Jacobian-free) adjoint; parallel reverse-AD transposed-Jacobian product	Gradient error 8.95% at np=4 scotch vs 0.00054% at np=4 simple 4x1x1 on the same mesh, same script, same container. Cross-residual of the converged scotch adjoint under the serial operator: 3.288×10^2 of $\ b\ $, against a 1.14×10^{-4} serial floor	Mechanism named at subsystem level; not traced to a line. Report drafted, not filed . No prior report anywhere (63 searches, 10 venues)
D2. Degenerate-branch rotation guard	IDWarp 2.6.2, src/utls/vec-torUtils.f90:58 , reverse mode vectorUtils_b.f90:123-128	warpDeriv discards the surface-normal-rotation term exactly whenever current normals equal reference normals — the state of every gradient evaluation. 210%/213% (upstream’s own test), 634% sign flip, 207%/122% sign flips; all collapse to FD-truncation level under a four-line analytic-limit patch, primal md5-identical	Root cause confirmed by repair. Matches open upstream issue mdolab/idwarp#57 (2021-07-14, zero comments). Comment-on-#57 prepared, not filed
D3. ILU conditioning wall	DAFoam v5, DALinearEqn.C: 266-267 : ASM sub-block PC hard-coded to PCILU	PetscConvergedReason: -9 (DIVERGED_NANORINF) at iteration 0 on a 21,000-cell closure case becomes reason 2 in 667 iterations under complete LU in the sub-blocks; the resulting 21,000-component field-inversion gradient verifies against central differences at 0.085%, 0.059%, 0.199%	Fixed locally by a one-hunk, default-off source patch. PETSc-sanctioned remedy; PR-ready in principle, not filed

Table 1: The three findings. Sources: Sections 2, 4, 5 respectively.

2 The decomposition-dependent adjoint

2.1 The phenomenon

The case is A4, a coarse Ahmed body: 2,777 cells, snappyHexMesh-class grid with refinement interfaces, DASSimpleFoam, kOmegaSST, one FFD shape design variable. The instrument is OpenMDAO’s `check_totals`, whose finite-difference column is computed *per run, inside that run’s own check_totals* — not computed once and reused. Only the decomposition varies across the rows of Table 2; verified by `diff` of every run directory against the source case (the np runs carry a byte-identical `runScript.py`; the **simple** runs differ by exactly one inserted `decomposeParDict` daOption line; the tolerance run by exactly the `gmresRelTol` value), and by reading the `Decomposition` method line back out of each log.

Four properties of this table decide everything that follows.

1. **The primal does not move.** Converged baseline C_D spans 0.15296979–0.15297237 across *all* configurations — invariant to five significant figures. np=1 and np=4 **scotch** differ by 5.5×10^{-7} absolute, 3.6×10^{-6} relative: separately reconverged SIMPLE solves at `primalMinResTol` 1e-4, i.e. reconvergence noise, four orders of magnitude below the 8.95% gradient shift. The **simple** arm — the arm with the *correct* gradient — differs from np=1 by *more* than **scotch** does.
2. **The finite difference does not move meaningfully.** The FD column spans (2.4178–2.4265)e-01, a 0.36% spread, against an 8.95% analytic excursion.
3. **Rank count is not the trigger.** np=2 **scotch** sits at the np=1 floor (0.26%); np=3 is catastrophic in one direction (6.05%, analytic high) and np=4 in the other (8.95%, analytic low); np=4 **simple** 4x1x1 is the cleanest configuration in the table. What changes between the dirty and clean rows is the *shape and orientation of the cut*.

configuration	log	analytic	FD (same run)	rel. err	
np=1, stock	a4_np1_stock.log	2.3965e-01	2.4232e-01	1.10%	
np=1, patched	a4_np1_patched.log	2.4150e-01	2.4232e-01	0.34%	
IDWarp					
np=2 scotch	a4_np2_patched.log	2.4118e-01	2.4182e-01	0.26%	
np=3 scotch	a4_np3_patched.log	2.5641e-01	2.4178e-01	6.05%	analytic high
np=4 scotch	a4_np4_patched.log	2.2086e-01	2.4258e-01	8.95%	analytic low
np=4 scotch,	a4_np4_	2.2086e-01	2.4258e-01	8.95%	811 iters, reason
gmresRelTol 1e-10	tol1.0e-10.log				2
np=4 simple 4x1x1	a4_np4_	2.4220e-01	2.4220e-01	0.00054%	
	simple4x1x1.log				
np=4 simple 1x4x1	a4_np4_	2.4379e-01	2.4265e-01	0.47%	
	simple1x4x1.log				
np=4 simple 4x1x1,	a4_np4_simple411_	2.4037e-01	2.4220e-01	0.76%	
stock	stock.log				
np=4 scotch, stock	a4_h1e-3_stock.log	2.1821e-01	2.4258e-01	10.04%	= published

Table 2: The verified phenomenon: $dC_D/d(\text{shape})$, analytic against each run’s own finite difference. Every cell re-extracted from the run logs by an adversarial supervisor sweep, VERIFICATION_A4_decomposition_supervisor_sweep.md (27D25762), Axis 1. Logs under /home/ubuntu/certonomous-runs/W4-a4-stepsweep/. “patched” denotes a locally patched IDWarp removing the independent defect of Section 4; the decomposition effect exists stock too, as the last two rows show.

4. **The solve is not the problem.** Tightening gmresRelTol from 1e-6 to 1e-10 costs 811 GMRES iterations instead of 719 and changes the answer by nothing. The np=4 **scotch** solve reports PetscConvergedReason: 2 at final true residual 1.749572×10^{-7} , and DAFoam’s own solver configuration makes that literal: DALinearEqn.C sets KSPSetPCSide(PC_RIGHT) and KSPSetNormType(KSP_NORM_UNPRECONDITIONED), so “converged in the unpreconditioned norm” means what it says.

This is the signature of a *converged wrong answer*. It is also the signature that makes the defect survivable in production: nothing in the solver’s own telemetry is anomalous.

2.2 The discriminators

Four candidate carriers were named in advance and executed cheapest-first (DISCRIMINATORS_A4_decomposition_mechanism.md; run artifacts under /home/ubuntu/certonomous-runs/W4-a4-discriminators/, with per-arm directories, ledger.txt, runScript_w4.py, build_maps.py, analyze_mats.py).

M3(a) — the primal, exonerated at zero cost. Point 1 above, plus this session’s own arms: baseline C_D 0.1529738 / 0.1529749 / 0.1529732 (np1 / scotch / simple), and mapped primal *state vectors* agreeing across decompositions at 2.0×10^{-3} / 2.8×10^{-3} relative — with the duplicated processor-face ϕ copies agreeing to 2.7×10^{-15} , machine precision. That last number does double duty: it validates the permutation used by every measurement below.

M3(b) — the mesh-derivative chain, exonerated by measurement. The dC_D/dX_v capture hook (a real reverse seed on DAFoamWarper) was run at all three decompositions with the warp finite-difference directional derivative $dX_v/d\text{shape}$ computed identically in each arm:

arm	$\langle dC_D/dX_v, dX_v/ds \rangle$	framework	compute_totals	agreement
np=1	2.414994844096177e-01	2.414994857859927e-01		5.7e-09
np=4 scotch	2.208588587555006e-01	2.208588593817390e-01		2.8e-09
np=4 simple 4x1x1	2.422030559126777e-01	2.422030573188107e-01		5.8e-09

The hand composition equals the framework total to 6×10^{-9} in *every* arm: the warp/DVGeo leg composes exactly at np=4 too. The warp direction itself is decomposition-invariant — dXv/dshape mapped point-by-point via `pointProcAddressing` matches np=1 at **0.0e+00** in both np=4 arms. What differs is the captured dCD/dXv seed: mapped to np=1 point ordering, `scotch` differs from np=1 by **38.4%** in norm, `simple` by 3.1%. The IDWarp/DVGeo leg is exonerated; dCD/dXv is wrong because the ψ it contracts is wrong.

M2 — coloring and preconditioner, exonerated by architecture and by measurement.

Both `dRdWTPC` and the full assembled `dRdWT` were dumped in all three arms and permuted to np=1 ordering. The diffs do not correlate with the defect: full-`dRdWT` Frobenius diff 4.8×10^{-2} (`scotch`) against 5.4×10^{-2} for `simple` — *the FD-exact arm*; PC diff 3.7×10^{-4} against 3.7×10^{-3} . This is a recorded negative, and the reason it is uninformative is itself measured: at np=1 the assembled `dRdWT` applied to ψ already disagrees with the AD product by $230 \times \|b\|$. The assembled matrix is a crude finite-difference object; only the preconditioner is built from it, which is precisely why the KSP operator is matrix-free. Diff operator *actions*, not operator *entries*.

Architecturally the exoneration is stronger still: in the Jacobian-free path the coloring machinery builds only `dRdWTPC`, and a wrong preconditioner cannot move a right-preconditioned GMRES solution that converged in the unpreconditioned norm (`DALinearEqn.C:170,313`) — which this solve did.

M1 — the cross-residual, which convicts the operator. Each arm’s converged adjoint ψ was dumped and permuted to np=1 global state ordering by an *exact integer map* built from `cellProcAddressing` / `faceProcAddressing` (sign carried for flipped faces) plus DA Foam’s own `AdjointIndexing` dumps (`writeJacobians: ["adjointIndexing"]`) — no coordinate matching anywhere in the construction. The true residual $\|A^T\psi + b\|$ was then evaluated under the *serial* AD operator via `calcJacTVecProduct`.

ψ from	$\ A^T\psi + b\ $ under np=1 operator	ratio to $\ b\ = 0.1826$	$\ \psi\ $
np=1 (control)	2.083e-05	1.14e-04	3.277e-02
np=4 <code>scotch</code>	6.004e+01	3.288e+02	5.411e-01
np=4 <code>simple</code> 4x1x1	1.711e-02	9.37e-02	1.814e-02

Table 3: The cross-residual. See Section 2.4 for the provenance of these values, which is not what the original record’s caption claimed.

Three decisive facts follow.

1. **The `scotch` ψ does not remotely satisfy the serial system** — $329 \times \|b\|$, 3.3 million times the np=1 floor — while it satisfies its *own* `scotch`-evaluated operator, same instrument, same API, at 1.2×10^{-3} relative, and the KSP that produced it reports final true residual 1.7×10^{-7} (719 iterations, reason 2). Same vector, two operators, answers five orders apart: **the operators differ**.
2. **The right-hand side is not the carrier.** $b = \text{d}C_D/\text{d}W$ mapped across decompositions is invariant to 4.8×10^{-6} (`scotch`) and 6.9×10^{-6} (`simple`) relative.
3. **$\|\psi\|$ is inflated 16.5× under `scotch`** (0.541 against 0.033 np=1 and 0.018 `simple`) at identical $\|b\|$. The extra content sits in the pressure (0.52 against 0.031) and z -momentum (0.117 against 0.005) adjoint blocks.

Confounds closed. (a) *Linearization state.* The cross-residual was re-evaluated with the np=1 operator linearized at the `scotch` arm’s own mapped state: `scotch` reads 6.004094×10^1 , unchanged to six digits, while the np=1 control degrades only from 1.14×10^{-4} to 1.0×10^{-3} .

The state confound therefore contributes at the 10^{-3} level, five orders below the signal — and the **simple** arm’s state differs *more* from np=1 than **scotch**’s does (2.8×10^{-3} against 2.0×10^{-3}) while its residual is 3,500 times smaller. (b) *Duplicated- ϕ averaging in the ψ map*. Replacing the average by either copy moves the residual by at most 9.1×10^{-3} , four orders below the signal. This bound was computed with the assembled matrix — the instrument M2 discredits — which the original record did not disclose; the supervisor sweep found it and reproduced the number exactly as $\|A_{\text{asm}}(\psi_{\text{copy}} - \psi_{\text{avg}})\|$. It is immaterial by margin: even inflated by the full $230\times$ assembled-vs-AD discrepancy the bound stays roughly $30\times$ below the 60.04 signal, and the duplicate copies disagree by at most 3.6×10^{-4} to begin with. The confound stays closed; the instrument should have been flagged.

2.3 The objection this measurement must survive

A knowledgeable reader will ask: is the **scotch** operator perhaps the *exact* adjoint of a different-but-equivalent parallel discretization? If decomposition changed the discrete residual itself at processor boundaries, the **scotch** adjoint could be consistent with *its* residual while differing from serial, and its 1.2×10^{-3} self-consistency would even fit that story.

It does not survive the finite-difference column. The FD in Table 2 is computed per run, inside each arm’s own `check_totals`. An adjoint that were the exact adjoint of an equivalent parallel discretization would match the finite difference of *that* discretization. The **scotch** analytic $2.2086\text{e-}01$ disagrees by 8.95% with its own decomposition’s FD of $2.4258\text{e-}01$. Combined with the fact that OpenFOAM processor patches evaluate the same interior face fluxes — no legitimate $O(1)$ residual-function difference is available — and that b maps invariantly at 4.8×10^{-6} , the supported conclusion is the strong one: **the parallel operator is not the transpose Jacobian of the residual it claims to differentiate.**

This gap was not acknowledged in the original mechanism record; the supervisor sweep found it and closed it from the prior record (`VERIFICATION_A4_mechanism_supervisor_sweep.md`, 8E0A08BC, Axis 4). It is stated here in the body, not in a footnote, because it is the first objection a maintainer will raise.

2.4 Sign-convention provenance, stated against ourselves

The values in Table 3 appear in no log, and the original record’s caption cited a log that contradicts them on its face. The instrument (`runScript_w4.py`, tasks `w4_dump` / `w4_crossres`) computes `res = Atpsi - b`, whereas the system OpenMDAO seeds is $A^T\psi = -b$. Every arm’s log therefore prints the sign-degenerate `ratio=2.000000e+00` for its own-operator check, and none of the record’s own-operator numbers — the 1.14×10^{-4} floor, the 1.2×10^{-3} self-consistency, the 2.083×10^{-5} — appears in any log.

The tabulated values are exact offline sign corrections, `res + 2b`, recomputed from the dumped vectors (`w4x_res_*.npy`, `w4_Atpsi`, `w4_b`). The correction is exact, and the adversarial sweep reproduces every digit of it three independent ways:

- **Recomputation from the dumps** with the sweep’s own scripts: np=1 1.140697×10^{-4} , **scotch** 3.288078×10^2 (absolute 6.004094×10^1 , max entry 42.25), **simple** 9.367397×10^{-2} .
- **An independently rebuilt permutation**, constructed from `cellProcAddressing` / `faceProcAddressing` and the `AdjointIndexing` dumps by the sweep’s own parsers, bit-for-bit identical to the session’s map in `tgt` and `sgn` — and validated on a channel the original session never used: the `AdjointIndexing` dumps carry cell and face centre coordinates, and mapped np=4 coordinates match np=1 coordinates at **max** $|\Delta xyz| = 0.0$ over all 26,149 (**scotch**) and 26,227 (**simple**) entries. Every entry of the map is thereby validated exactly, independent of any solved field.

- **A fresh serial container run:** a new case directory staged from the same source, fresh serial primal, `w4_crossres` applied to ψ vectors mapped by the sweep’s own permutation. Raw `scotch` ratio 3.288139×10^2 ; sign-corrected 3.288078×10^2 ; np=1 control 1.140697×10^{-4} ; simple 9.367397×10^{-2} (`w4-a4-verify-audit/audit_crossres.log`, 9 s, 0.30 core-min).

The defensible citation for Table 3 is therefore the dumped vectors plus the stated sign correction — not `d_crossres.log`. Anyone reproducing this protocol should fix the convention in the instrument rather than offline; `w4_crossres2`, written later, has the sign right and its log matches its numbers.

2.5 Localization

The 60.04 cross-residual is not spread over the domain. By state group, **59.78 of the 60.04 norm sits in U_0 (x -momentum) rows**. By cell, entries with $|r| > 0.5$ number 15, of which **13 sit on scotch partition-interface cells and 2 at graph distance one** — none farther. All 15 sit on `cellLevel` 0 cells, away from the mesh’s refinement interfaces. The two worst (42.25 and 42.24) sit at $y = -0.1947$ and $y = +0.1949$: the y -normal cut signature.

Three structural facts sharpen this.

- **It is not a many-rank corner effect.** The worst cells each touch exactly *one* foreign rank through exactly *one* processor face; cells touching two to four foreign faces carry *smaller* residuals.
- **Cut orientation, not cut jaggedness alone, sets the magnitude.** The hierarchy in Table 2 is orientation-ordered: x -normal planar slabs 0.00054%, y -normal planar slabs 0.47%, `scotch`’s jagged mixed-orientation boundary 8.95% — and the worst cross-residual entries sit on y -normal processor faces.
- **The same instrument on the clean arm is five thousand times quieter.** Applied to the simple $4 \times 1 \times 1$ ψ , the serial operator maxes at **0.008439** on that arm’s own interface cells (9 of its top 10 entries sit on them) — $5,006\times$ below `scotch`’s 42.25. Had `simple`’s residual also been huge, the test would have proven nothing.

The hanging-node hypothesis, refuted backwards. The natural first guess on a refinement-interface mesh is that partition cuts crossing hanging nodes are the trigger. The supervisor sweep re-classified all 7,843 internal faces itself, from `owner/neighbour/cellLevel` and the archived `cellDecomposition` maps: 456 refinement-interface faces, of which `scotch` cuts **4** (0.88%) and `simple` $4 \times 1 \times 1$ cuts **68** (14.91%). *The decomposition that cuts seventeen times more refinement interfaces is the accurate one.* The hypothesis is refuted, not merely unconfirmed.

2.6 The mechanism, named — and what is not claimed

Named. Under decomposition, DA Foam v5’s reverse-mode AD product $(\partial R / \partial W)^T v$ — the global-tape shell operator the adjoint KSP iterates (`initializeGlobalADTape4dRdWT / dRdWT-MatVecMultFunction`, `DASolver.C:1364ff`) and the `calcJacTVecProduct` API that reverses the same recorded computation — differs from the true transposed Jacobian of the discrete residual at partition-interface cells, by amounts that depend on the cut’s shape and orientation.

The two parallel AD paths sit *together* on the wrong side: they agree with each other to 1.2×10^{-3} of $\|b\|$ while both sit 3.3×10^2 of $\|b\|$ from the serially evaluated truth. The defect is therefore common to the recorded reverse computation, not to one call site. GMRES then solves the wrong system exactly; the adjoint inherits $O(1)$ error along the directions the wrong operator mislabels (hence the $16.5\times$ norm inflation); and the gradient contraction $\psi^T \partial R / \partial X_v$ delivers the 8.95%. Every observed symptom follows: converged wrong answer, tolerance invariance, decomposition dependence, primal/FD/RHS invariance, and the near-invariance of the lab’s conformal-mesh cases.

Not claimed. The defective source line. The candidate surface is the reverse-AD handling of inter-processor halo exchange inside the recorded residual evaluation — the CoDiPack tape over OpenFOAM processor-boundary updates — and pinning the line requires an instrumented rebuild of `libDASolver.so`. What *is* pinned: the subsystem, the spatial signature (interface-cell x -momentum rows), the orientation dependence, and the discriminating instrument that any candidate fix must satisfy.

A continuum, not a binary. The FD-exact `simple 4x1x1  $\psi$` still leaves 9.4×10^{-2} of $\|b\|$ under the serial operator — roughly $800\times$ the `np=1` floor — while its gradient is correct to 0.00054%. Every `np=4` operator tested differs measurably from serial; `scotch`’s jagged cut is the catastrophic point on a continuum, not a separate species.

2.7 A usability finding recorded on the way

The last and most expensive discriminator — run with coloring disabled — could not deliver its number, and the reason is worth reporting to upstream in its own right.

`adjUseColoring: False` crashes the **v5 mphys Krylov path by construction**: `mphys_dafoam.solve_linear` skips `runColoring()` when the option is `False`, but `calcDrdWT` unconditionally calls `readJacConColoring()`, which reads a file that was never written and aborts in `DAColoring::validateColoring`. The static chain was re-confirmed in the shipped container by the supervisor sweep: `mphys_dafoam.py:458` skips `runColoring()` unless `adjUseColoring` is `true`; `DA-Solver.C:1043` calls `readJacConColoring()` unconditionally; and `DAColoring::validateColoring` (beginning line 931) reaches its `FatalErrorIn ... abort` at approximately `DAColoring.C:1021`.

The identity-coloring branch (one colour per global column) is reachable only through `runColoring()`, so the arm was rerun with an explicit `runColoring()` call before `compute_totals`: identity coloring accepted, `nJacConColors: 26149`, preconditioner partials computed by pure per-column finite difference with no colour compression. **The forced path is memory-unbounded.** On this 2,777-cell case (colored path peaks under 2 GiB) the first forced attempt thrashed an 8 GiB container cap and was killed at 701 s; a second at a 20 GiB cap completed all 26,149 column evaluations in 258 s and then exhausted 20 GiB in the post-assembly stage — consistent with per-column FD columns of an elliptically coupled residual being nearly dense — and was killed thrashing at 591 s.

The pre-registered prediction for this arm (“the gradient will not move, because coloring builds only the preconditioner”) is scored **NOT SCORED**: the discriminator never met its measurement. The coloring exoneration does not rest on it; it rests on architecture plus M1, as set out above.

2.8 How far the defect reaches

The trigger condition is bracketed by measurement rather than asserted from one case. Four cases in this lab are already on record as decomposition-invariant — A1 (4,032-cell structured airfoil, `np=1` vs `np=4`: relative error 3.795529×10^{-4} vs 3.807681×10^{-4}), A2 (38,304-cell conformal wing), A5 (4,800-cell conformal U-bend, six 27-component gradient groups all invariant at norm level, worst group-norm difference 3.8×10^{-4}) — and every clean case is conformal while the only dirty case is the only snappy-refined mesh tested. That confound is the target of a pre-registered breadth matrix, `DEFECT_REACH_decomposition_cases.md` (pre-registration committed at 99F5D41D before any arm ran).

Status at the time of writing (2026-08-05). All arms have run — the ledger at `/home/ubuntu/certonomous-runs/W4-defect-reach/ledger.txt` records thirteen entries from the Ahmed-35 mesh generation through the contingent N7 cross-residual — but the results section of the reach record is still being written and its predictions are not yet scored. One result has been relayed from that work in progress: **N6, CBFS, is decomposition-invariant at 1.1×10^{-4}** ,

arm	configuration	pre-registered prediction
N1	A4, np=4 simple 1x1x4 (z-normal slabs)	rel. err in [0.05%, 1.5%]: planar slabs of any orientation benign-to-moderate
N2	A4, np=4 simple 2x2x1 (4-rank corner line)	rel. err $\leq 1\%$. Decision rule: if $> 2\%$, “scotch-style jaggedness required” is REFUTED
N3	Ahmed-35, a <i>new</i> snappy-refined case built by A4’s own recipe, np=1	control: rel. err $\leq 1.5\%$
N4	Ahmed-35, np=4 scotch	headline: defect appears, rel. err $\geq 2\%$, analytic differing from N3 by $\geq 2\%$ while FD stays within $\sim 0.5\%$
N5	Ahmed-35, np=4 simple 4x1x1	clean: rel. err $\leq 1\%$ regardless of N4
N6	CBFS, 21,000-cell conformal blockMesh, 21,000 beta DVs, simple 4x1x1 vs the scotch record arm	decomposition-invariant: $\leq 1\%$ per component at the three recorded FD cells, norms to $\leq 2\%$
N7	Ahmed-35 cross-residual (contingent on N4)	$\geq 10 \times \ b\ $ under the np=1 operator, on momentum rows of interface cells

Table 4: The pre-registered reach matrix. Predictions committed before any solver ran; scoring is HELD / NOT HELD / NOT SCORED exactly as written, with no post-hoc bands.

consistent with the pre-registered prediction and with the conformal-mesh pattern. *The remaining arms are marked in progress here deliberately; this document will not pre-empt a pre-registered scoring with numbers read out of raw logs.*

Two record facts the matrix already leans on need no new run. First, the np=2-clean / np=3-dirty contrast (0.26% vs 6.05%) rules out rank count as the trigger by itself. Second, the “cut through the refinement region” discriminator is already on record backwards, as set out in the localization above.

3 Why it was never caught

Two independent bodies of evidence answer this: the published verification protocols of the toolchain’s own method papers, and the community record.

3.1 The papers’ own protocols

Three method papers were read in full and their verification protocols tabulated (DAFOAM_PAPERS_VERIFICATION_PROTOCOLS.md, BF6AC53B; reading notes with per-claim source tables under demo-output/website/campaign/).

- He, Mader, Martins, Maki, “DAFoam: An Open-Source Adjoint Framework for Multidisciplinary Design Optimization with OpenFOAM,” *AIAA Journal*, doi:10.2514/1.J058853.
- He, Mader, Martins, Maki, “An aerodynamic design optimization framework using a discrete adjoint approach with OpenFOAM,” *Computers & Fluids* 168 (2018) 285–303, doi:10.1016/j.compfluid.2018.04.012.
- Kenway, Mader, He, Martins, “Effective Adjoint Approaches for Computational Fluid Dynamics,” *Progress in Aerospace Sciences*, doi:10.1016/j.paerosci.2019.05.002.

Three facts fall straight out of Table 5.

1. **No paper ever varies the decomposition of anything.** Decomposition is not an experimental axis anywhere in the corpus — not at fixed process count, not across process

paper	adjoint operator verified	case / mesh class	cells	decomposition	result
C&F 2018	v1 explicit FD-coloring Jacobian	Ahmed body 25°, snappy- HexMesh	~1M	unstated	avg <0.1%, worst 0.252%
AIAA J 2020	v1 explicit FD-coloring Jacobian (Jacobian-free explicitly deferred to future work)	ADODG Case 3 wing, structured hex	102,912	unstated ; “decompose”/ “processor”/“scotch” absent from the paper	avg <0.1%, worst flow row 0.195%
PAS 2019	Jacobian-free reverse AD (operator overloading) — the v2+/v5 architecture family	ADODG Case 3 wing, structured hex	102,912	serial. np=1, stated twice, by design	10-digit agreement
PAS 2019	Jacobian-free source- transformation AD — <i>ADflow</i> , not <i>DAFoam</i>	CRM wing / wing-body-tail	3.6M / 10.4M	block-based, fixed (48 / 96 cores)	11 / 6 digits

Table 5: Every gradient-accuracy verification in the method-paper corpus.

counts. The two papers that verify DAFoam’s v1 operator in (implied) parallel never state what the decomposition was.

2. **The operator our defect lives in has exactly one published accuracy measurement, and it is serial.** The survey states the reason in its §5.1 — running on one CPU core “allows us to isolate the impact of parallel communication on the performance” — and confirms the gap in its conclusions: “*We do not have scalability data for the Jacobian-free (operator overloading) and the full-code AD (operator overloading) options because we run the adjoint computation only in serial in the ADODG Case 3 (Sec. 5.1).*”
3. **No verification anywhere combines a refinement-interface mesh with a stated graph partitioning.** The one snappy-mesh verification — C&F 2018’s Ahmed body, our defect’s own geometry family — is of the v1 explicitly FD-assembled Jacobian, an architecture in which a tape-recording defect cannot exist, at an unstated decomposition.

A published claim, corrected against ourselves. An earlier draft of our own upstream report asserted that “the toolchain’s journal paper reports average adjoint derivative error under 0.1% at up to 1536 cores” and that our measurement therefore contradicts a peer-reviewed number. Reading the paper in full showed that sentence to be *wrong twice*. The abstract joins two disjoint experiments with an “and”: the 1536 cores is a runtime-only table on a 10.1M-cell structured mesh; the <0.1% is a separate ~103k-cell accuracy study at an unstated processor count. **No accuracy number at any stated core count exists in that paper.** And both <0.1% figures attach to the v1 explicit-FD-Jacobian architecture — the same paper pairs them with “Implementing the Jacobian-free approach in DAFoam ... will be conducted in the future work.” The 0.1% scale is the FD-partials error floor, not a parallel-consistency scale.

This *strengthens* the case rather than weakening it: the report does not contradict a published measurement, it fills a hole the survey’s own conclusions declare.

No archival method paper covers the shipped architecture. The DAFoam publications page lists no framework paper after 2020; the mdolab/dafoam README’s citation section names exactly the two He et al. papers, tied to no version; a JOSS search finds nothing; Zenodo records 15636000 (v4.0.2) and 20045081 (v5.0.0) are software deposits, not papers. The v2.2 release note that shipped the Jacobian-free adjoint cites no paper for it either. *The v4/v5 rewrite we measured*

has no archival method paper at all; the citation trail for the shipped matrix-free adjoint ends at papers describing other architectures.

Was the defect class disclosed anywhere? Two quotes were fetched for exactly this question, and neither is a tape caveat. C&F 2018 §2.9 names interprocessor boundary-patch state updates as “*essential for accurately computing the adjoint derivative*” — for the v1 FD path, where they are handled by executing `U.correctBoundaryConditions()` per perturbation. The v5 tape must *record* that same coupling, and its interface-cell rows are exactly where our cross-residual concentrates. PAS 2019 §A.4 acknowledges that “*some code structures remain a challenge to differentiate, including pointers and MPI calls*”, and that ADflow’s parallel derivative correctness rests on manual assembly using properties “that the AD code cannot, in general, safely assume” — a statement about AD-across-MPI as a hazard class, made about ADflow’s development choices.

No paper states a halo-exchange tape caveat or a partition-consistency assumption for DAfoam’s operator-overloading path. There is no sentence a maintainer could point to as prior disclosure. What the quotes support is gentler and more useful: **the defect sits in a subsystem whose verification the papers explicitly scoped out, in a region their own earlier work flagged as accuracy-critical.**

3.2 The community record

A read-only novelty sweep ran **63 recorded searches across 10 venues** on 2026-08-04, with roughly 25 further pages read in full: the mdolab/dafoam issue tracker and Discussions, mdolab/id-warp issues and discussions, mdolab/pyofm, mdolab/MACH-Aero, mdolab/adflow, OpenMDAO/OpenMDAO, GitHub global search, web and scholar search, and the PETSc lists (LIAISON_NOVELTY_SWEEP_decomposition_defect.md, 3c74dc03). Nothing was posted; every URL cited in that memo was fetched in-session, and quotes are marked verbatim or via-summary.

Verdict: no prior report. No one has ever reported a DAfoam gradient that is converged and wrong under one decomposition and correct under another; any **scotch-versus-simple** dependence; any transpose-operator inconsistency; or anything of this class on a plain wall-bounded case without coupled patches. The word “**scotch**” **appears in zero issues and zero discussions in the project’s history**, and `getRotationMatrix3d` appears in zero issues or pull requests anywhere on GitHub.

What the record *does* contain makes the case sharper, not weaker:

Three consequences for how the finding should be presented upstream.

1. **The “known issue with coupled patches” reply is pre-answered.** Every upstream acknowledgement of parallel-wrong derivatives attributes the wrongness to a coupled patch (periodic, AMI), prescribes avoidance, and names no mechanism. Our Ahmed-body case has *no* coupled patches: it shows the same class of wrongness arising from ordinary processor boundaries — precisely the configuration upstream’s own lore calls safe.
2. **The “decomposition only affects convergence” reply is pre-answered.** #885/#952 concern convergence rate; our KSP *converges* (reason 2, true residual 1.7×10^{-7}) to a wrong solution, which is exactly what the cross-residual measurement discriminates.
3. **The triggering configuration is the default.** The docs make no parallel-consistency claim and give no decomposition guidance at all, while the shipped default is **scotch** (pyDAfoam.py lines 590–591, main at e77f0c0c, fetched raw 2026-08-04: `self.decomposeParDict = {"method": "scotch", ...}`).

The remaining gaps are not novelty gaps. They are, unchanged: a reproducer on a stock upstream tutorial rather than a lab-built case, and mechanism to a line.

thread	what it establishes
#946 (2026-02)	The strongest near-miss found anywhere. A user’s <code>check_totals</code> shows 93.92% analytic-vs-FD error on the PeriodicHill case; the maintainer replies, verbatim: “ <i>The PH case’ derivatives are not accurate when running in parallel. You have to run it in serial.</i> ” No mechanism offered, scoped to the periodic-boundary case
#379 (2022-12)	The same acknowledgement four years earlier — the periodic hill case “is known to have issues when running in parallel (because it has periodic boundaries; we haven’t gotten a chance to debug it yet)”, while other cases “can run in parallel without an issue”
#972 (2026-04)	Nearest <i>mechanistic</i> relative: cyclicAMI adjoint false-converges then produces 1e11–1e15 sensitivities; remedy is to force all AMI faces onto one rank. A coupled interface whose parallel adjoint treatment is wrong — but AMI patches, v4 assembly path, and garbage output rather than a cleanly converged 9%-wrong gradient
#885 (2025-09), #952 (2026-03)	Upstream’s clearest acknowledgement that decomposition affects the adjoint — but strictly its <i>convergence rate</i> . KAHIP partitioning improves the preconditioner; kahip support was added upstream 2025-12. Nobody in either thread checks the converged gradient <i>value</i> against decomposition
#101/#102 (2021-03)	The one time serial-vs-parallel gradient equality was actually checked upstream, it passed: the reported discrepancy was cell ordering in a written field, not the gradient. v1-era topology case
#914 (2025-11)	Documents upstream’s acceptance threshold: a 1.27% analytic-vs-FD deviation draws “the gradient accuracy is very acceptable for practical aerodynamic optimization”

Table 6: The community record on parallel adjoint correctness in DAFoam.

4 The IDWarp rotation guard

This defect is independent of Section 2 — it is geometric, serial, and lives in a different repository — but it was found in the same arc, it contaminated the same gradients, and it had to be removed before the decomposition measurements could be trusted. It is also the cleanest of the three: root cause named to a line, confirmed by repair.

4.1 The line

IDWarp **2.6.2** is the exact version in `dafoam/opt-packages:latest` (image digest `sha256:9d45679d55fd47f5ca7afd99cabb86c7c2729cf2acf34c438eb33af5290f07fc`); the installed `idwarp/*.py` files are md5-identical to the v2.6.2 tag of <https://github.com/mdolab/idwarp>, tag commit 647FD8FC.

The warp displaces each volume node by an inverse-distance weighted sum in which the weights W_i are *constant* with respect to the surface coordinates; all the surface dependence lives in a rotation matrix M_i and a displacement b_i , with $S_i = M_i \cdot (r - X_{u0,i}) + (X_{u,i} - r)$. **The rotation acts on the lever arm from the surface node to the volume node.** M_i is built at `src/modules/kd_tree.F90:1639-1640` as $R(n_0 \rightarrow n)$, by:

```

real(kind=realType), parameter :: tol = 1.4901161193847656e-08    ! line 44 (= sqrt(eps))
call cross_product_3d(v1, v2, axis)
call getMag(axis, axisMag)
! When axisMag is less than sqrt(eps), the acos 'arg' value will be
! exactly one which will give a nan in complex mode.
if (axisMag < tol) then                                           ! line 58 <<< HERE
    angle = zero
    axis = zero
    axis(1) = one

```

```

else
  axis = axis / axisMag
  vv1 = v1 / magv1 ; vv2 = v2 / magv2
  arg = min(one, vv1(1)*vv2(1) + vv1(2)*vv2(2) + vv1(3)*vv2(3))
  angle = acos(arg)
end if
Mi = Mi + sin(angle) * A + (one - cos(angle)) * C
! Rodrigues

```

The map $M_i(n_0, n)$ is perfectly smooth at $n = n_0$, with the finite, non-zero derivative $dM_i = [n_0 \times dn]_{\times}$. But *this parameterisation* — normalise the cross product, take `acos` of the dot product, apply Rodrigues — has a removable coordinate singularity there: `axis` is 0/0 and $d(\arccos)/d(\arg) \rightarrow \infty$. The guard exists to stop that becoming a NaN, and the source comment says so. It replaces the singularity with `angle = 0`.

What Tapenade does with the guard. In the generated reverse file `src/adjoint/outputReverse/vectorUtils_b.f90:123-128`, when the guard branch was taken the reverse sweep *zeroes every adjoint on the path back to v2*; the subsequent `GETMAG_B` and `CROSS_PRODUCT_3D_B` calls therefore propagate exactly zero, and $dM_i/d(\text{normals}) = 0$ exactly. Tapenade is not at fault: it differentiated the branch that was taken. The defect is that the branch taken is a *constant* where the true function has a non-zero first-order term.

The guard fires with certainty at every gradient evaluation. `getMag` regularises with $+10^{-30}$ under the square root. At an undeformed baseline `normals == normals0` bit-for-bit (same code, same inputs), so `axis =  $n_0 \times n = 0$` and `axisMag =  $\sqrt{10^{-30}} = 10^{-15} < 1.4901 \times 10^{-8}$` . **Branch 0, guaranteed, at every non-corner surface node.** A `check_totals`, a first design iteration, and every gradient probe are evaluated at exactly that state. The finite difference, meanwhile, perturbs the surface by h and tilts adjacent faces by $\theta \sim h/L$, far above the threshold — so the FD sees a first-order rotation term the AD has set identically to zero. *That difference is the gap.*

4.2 Narrowing the claim, against ourselves

The obvious statement — “the rotation term is mis-differentiated” — is too broad, and measurement refuted it. Across five of IDWarp’s own regression meshes, run with the project’s own displacements:

mesh	deformation	worst err, rotations ON	worst err, rotations OFF	$\ dX_s\ $ on vs off
symm_block (inflate_cube)	axis scaling	212.6%	1.06e-05%	identical
o_mesh	shear sweep	1.7e-05%	5.8e-03%	6.42e+03 / 9.46e+02
co_mesh	shear sweep	3.9e-05%	5.1e-05%	7.27e+04 / 2.16e+03
sym_mesh	shear sweep	6.9e-06%	9.1e-04%	8.85e+04 / 2.45e+03
onera_m6	shear sweep	1.26%	9.7e-07%	1.69e+04 / 3.06e+03

On the three shear cases the deformation genuinely rotates normals, the guard does *not* fire, the AD *does* change when rotations are switched off — and with rotations **on** those cases are clean at $10^{-5}\%$ to $10^{-6}\%$. **When the rotation branch is live, warpDeriv is correct.** The catastrophic failure is confined to $n \approx n_0$. This is a sharper and more falsifiable statement than the one we started with, and it is the correct one.

The capstone that could have refuted it. If the failure is the degenerate branch specifically, then moving a case off its undeformed baseline should make the guard stop firing, at which point `warpDeriv` must become sensitive to `useRotations` and get dramatically more accurate. With all 27 shape design variables pre-set to 0.02 on the U-bend case, the two sign flips vanish and the

worst error falls from 207.0% to 8.2% with nothing else changed — and the fingerprint confirms the branch is now live: $\|dX_s\|$ is $1.495168856286067 \times 10^3$ at the undeformed baseline whether rotations are on or off (*identical* — branch dead), and $1.566420804338303 \times 10^3$ vs $1.495168856286067 \times 10^3$ pre-deformed (*differs* — branch live). Had the mechanism been “rotations are mis-differentiated”, this experiment would have failed flatly.

4.3 Why upstream’s CI cannot see it

Four independent mechanisms, each verified against primary source.

1. **The assertion is missing.** `tests/test_USMesh.py:86` calls `mesh.verifyWarpDeriv(...)` and *discards the result*. Nothing is asserted on it — which is exactly what upstream issue #57 says.
2. **The only thing asserted is blind by construction.** The regression check is `handler.root_add_val("... Sum of dxs:", val, tol=1e-8)`. Across five meshes the rotation term contributes *exactly zero* to that sum (identical to 0–1 ulp) while changing $\|dX_s\|$ by factors of 5 to 34. Five for five, upstream’s check cannot detect a change that alters the derivative vector by a factor of 34.
3. **A dot-product test cannot detect it either, and is not run.** `examples/structured/-dotprod.py` lives in `examples/`, not `tests/`. And a dot-product test only proves forward-AD and reverse-AD are mutual transposes: if both linearise a *different* function than the warp, they do so identically and the dot product is still zero.
4. **The verification tool defaults to a random seed.** `UnstructuredMesh.py:460: def verifyWarpDeriv(self, dxv=None, ..., randomSeed=314)`. A random direction contracts the error against a direction it barely overlaps (0.32–1.30% where the real objective seed reads 207%, flipped).

Not a documented approximation. No documentation, release note, docstring, or source comment qualifies `warpDeriv` as approximate. The docs say only “Compute the warping derivative (dX_v/dX_s^T)*Vec”; the `warpDeriv.F90` header says only “Compute the following matrix vector product: $dX_v/dX_s^T * Xvbar$ ”. Both present it as the exact transpose Jacobian. *One disclosed gap:* the primary IDWarp reference (Secco, Kenway, He, Mader, Martins, *AIAA J.* 59(4):1151–1168, 2021, doi:10.2514/1.J059491) could not be retrieved — five umich URLs return 403 including the authors’ own bibliography link, and OpenAlex (`is_oa: false, oa_status: "closed"`) and Semantic Scholar independently report no open copy. No claim is made about what that paper says, and the “undocumented” characterisation rests on code, docs and release notes only.

4.4 The upstream issue this answers

`mdolab/idwarp#57`, “`inflate_cube` test appears to fail”, opened 2021-07-14, labelled **bug**, **open**, **zero comments**, **five years untouched**. Its timeline contains exactly one event: the label, applied 8 seconds after creation. Its author wrote: “*the output from this test appears to show large errors in some of the derivative terms. However, these error values are not actually used as a unit test and so the entire test suite still passes*”, and “*Given how weird the FD values are for the two terms with big errors, it may be that the issue is in those rather than the AD values, but it’s hard to see how that could be the case.*”

We reproduced the issue exactly — IDWarp alone, no DA Foam, no OpenFOAM, no CFD, using IDWarp’s own test mesh, options, displacement and verification routine — and answered its open question adversarially. A step-size study on the two failing degrees of freedom gives FD values stable to six significant figures across three decades ($h = 10^{-4}$ to 10^{-6}), with subtractive

roundoff appearing only at 10^{-8} . **The finite difference is converged; the AD is the wrong one.**

Why degrees of freedom 0 and 3 and not 1, 2, 4, 5: `inflate_cube` scales an axis-aligned block by (1.1, 1.2, 1.3), which leaves every face normal *exactly* unchanged, so the guard fires even at the deformed state. Perturbing a node *along* its face normal tilts adjacent faces at first order and switches the rotation term on; perturbing it *in* the face plane does not. That is a clean instance of our own predicted-clean control, appearing unbidden inside upstream’s five-year-old data.

4.5 The patch, and what it proves

The derivation was written *before* the code, and every sign was checked by a controlled experiment with a hand-computed known answer before any system-level run. With $u_1 = v_1/|v_1|$, $u_2 = v_2/|v_2|$: $\sin \theta [k]_{\times} = [u_1 \times u_2]_{\times}$ exactly, and $(1 - \cos \theta)[k]_{\times}^2 = O(\theta^2)$, so $M_i = I + [u_1 \times u_2]_{\times} + O(\theta^2)$ and, at $u_2 = u_1$,

$$dM_i = [w]_{\times}, \quad w = \frac{v_1 \times dv_2}{|v_1| |v_2|} \quad (\text{forward})$$

$$v_{2b} += \frac{\text{axial}(m_{ib} - m_{ib}^T) \times v_1}{|v_1| |v_2|} \quad (\text{reverse, by the cyclic triple product})$$

$|v_1|$ and $|v_2|$ are already recomputed in the reverse routine’s forward sweep and are in scope inside the branch. **The whole fix is four lines of assignment in the branch .EQ. 0 block**, replacing nothing: the three zeroed intermediates stay zeroed, because the dummy-axis path they belong to really does carry zero; the patch adds the term that path lost. The diff is 2 files, +44 lines, all in `src/adjoint/output{Reverse,Forward}/vectorUtils_{b,d}.f90`.

acceptance test (stated before the runs)	before	after	verdict
Upstream <code>inflate_cube</code> (#57), rotations ON, DOFs 0 / 3	210.16% / 212.62%	8.56e-06% / 3.35e-05%	PASS
A1 NACA0012, real dCD/dXv seed, idx6 (leading-edge combo)	634.0%, sign flipped	5.54e-04%	PASS
A5 U-bend, pressure-loss real seed, idx8 / idx17	207.0% / 121.6%, both flipped	3.0e-06 / 7.0e-06 (relative)	PASS
Regression control: <code>o_mesh/co_mesh/sym_mesh/onera_m6</code> , guard dead	—	bit-identical verifyWarpDeriv output, every printed digit	PASS
Primal invariance: 310,284-coordinate warped grid	—	md5-identical (8fafe12f848a f490a5041c865112b5fb), max diff = 0.0	PASS

Table 7: Acceptance results. Note the *direction*: on #57 the AD moved to the already-converged FD ($2.87055669475 \rightarrow -115.875779229$ against an unchanged FD of -115.875789153) — the direction a genuine fix must move and a comparison-zeroing artifact could not produce. The in-plane DOFs 1, 2, 4, 5 are untouched to every printed digit.

On the A2 wing the patch improves three of four `check_totals` rows by 33.9x, 53.2x and 115.1x (CD/shape $1.713791 \times 10^{-2} \rightarrow 5.059114 \times 10^{-4}$; CL/shape $1.165163 \times 10^{-2} \rightarrow 2.189473 \times 10^{-4}$; CL/twist $1.120617 \times 10^{-2} \rightarrow 9.737824 \times 10^{-5}$) — and *degrades* the fourth, CD/twist, from 0.389% to 0.505%, both deep inside the pass band. Any one-line summary quoting “34–115x” must carry that footnote.

Two limits, stated plainly. First, **retuning the tolerance cannot fix this.** `tol` is exactly $\sqrt{\varepsilon}$, but the `acos(min(1, v1.v2))` construction is numerically flat on its own out to $\sqrt{2\varepsilon} = 2.1073 \times 10^{-8}$ rad: $1 - v_1 \cdot v_2 = \theta^2/2$ drops below ε , `arg` rounds to bit-exact 1.0, and `acos(1.0)` is exactly 0, guard or no guard. Measured at $\theta = 10^{-8}$: `arg` bit-exactly 1.0, $1 - \text{arg}$ bit-exactly 0.0. *The first thing a maintainer will reach for is foreclosed.* Second, **the AD’s zero is the exact derivative of the implemented function at that point.** The guard makes the warp genuinely flat in a ball of angular radius $\sim 1.5 \times 10^{-8}$ rad, and a finite difference taken entirely inside that ball returns exactly 0.0, agreeing with the AD to every digit. Tapenade is correct about the code as written. The defect is precisely that `warpDeriv` returns *the derivative of a flat spot that no optimizer can traverse and no finite difference can resolve* — reaching it needs $h < 1.5 \times 10^{-8}L$, and the step study already shows roundoff destroying the FD at $h = 10^{-8}$. Any filing should pre-empt the response “the code is differentiable and your step is too coarse.”

4.6 The second regime, and the fix that covers both

Away from the baseline, with `axisMag` just above `tol`, the guard does not fire and the derivative *is* computed — but through an `acos` whose argument has lost its information to cancellation. This regime is real, measured on ONERA M6 at $\sim 1.26\%$, and **survives the patch, exactly as predicted** before the acceptance runs. Its cause was *tested* rather than asserted, by repairing each candidate line in isolation:

θ	V0 shipped	V1 denominator only	V1b angle only	V2 both	V3 singularity-free
1e-06	6.658e-05	5.561e-05	3.661e-05	1.974e-10	1.538e-12
1e-07	3.998e-04	3.339e-04	2.199e-04	7.221e-10	1.710e-12
5e-08	1.161e-02	9.663e-03	6.436e-03	3.083e-09	2.229e-12
2e-08	5.286e-02	4.483e-02	2.801e-02	4.844e-09	1.068e-12
$\leq 1.49\text{e-}08$ (guard)	1.0	1.0	1.0	1.0	9.3e-13

Table 8: Relative error of the reverse routine against the true derivative of the smooth map. V1 is the single-line repair an earlier draft of our own record named as the carrier: it buys $1.0\times$ to $1.6\times$, i.e. it is indistinguishable from doing nothing. The pair buys 2.1×10^3 to 3.7×10^7 .

The dominant term is `angle = acos(arg)` in the *primal*: `arg = 1 -  $\theta^2/2$` loses the information at $\theta \sim \sqrt{\varepsilon}$ before `acos` is ever called, and the reverse inherits it. Neither line is sufficient alone and the two do not add. Only `angle = atan2(axisMag/(magv1*magv2), arg)` *together with* $\sqrt{1 - \text{arg}^2} \rightarrow \text{axisMag}/(\text{magv1} \cdot \text{magv2})$ removes the regime — and the substitute is free, because `axisMag` is already computed two lines earlier in the primal.

The recommended upstream fix is neither of our patches but the reparameterisation $R = I + [v]_{\times} + [v]_{\times}^2/(1+c)$ with $v = a \times b$, $c = a \cdot b$: a polynomial in v with the smooth coefficient $1/(1+c)$, giving I with the correct derivative at $a = b$, needing no threshold, and unable to NaN except at exact reversal — a genuine singularity of the problem rather than of the parameterisation. It is measured here at 6.3×10^{-13} to 2.2×10^{-12} across every angle tested *including inside the guard ball*, where every `acos`-based variant reads exactly 1.0. It is the only single change that covers both regimes.

Two caveats attach. A regime-2 repair **cannot** be primal-bit-identical — it changes `angle` in the forward warp (M_i moves by 1 ulp at $\theta = 1$) — so any such patch must be re-graded on the primal as well as on the gradient. And our shipped patch is a *proof of concept on the generated file*: upstream should fix the primal’s parameterisation and regenerate, not merge a hand-edit of Tapenade output.

4.7 Workaround, with its price named

`useRotations: False` in `meshOptions` restores gradient consistency to roundoff on all evidence here — it makes the warp exactly linear in the surface coordinates. It is not free: upstream

documents rotation interpolation as what “will help preserve boundary layer orthogonality” under large orientation changes, so mesh quality under large deformation is the price. **That trade has not been measured by this lab**, and no optimisation has been re-run under it. Recommending it as a default would be premature.

5 The conditioning wall, and the sub-LU remedy

5.1 The wall

On a 21,000-cell curved-backward-facing-step (CBFS) field-inversion case, the DAfoam adjoint linear solve fails at iteration 0 with `PetscConvergedReason: -9 (DIVERGED_NANORINF)` and printed iteration-0 residual $7.091590452305 \times 10^{-4}$. It fails identically under all five available `jacMatReOrdering` values — two produce `-9`, three produce `-3` flat stagnation, none converges — and the printed initial residual is identical to 13 digits across all of them, which is the tell that the failure is in the factorization, not the Krylov space.

5.2 The offline reproducer, which made every candidate a ten-second experiment

The dumped CBFS system (`dRdWTPC`, 210,592², nnz 13,710,468, plus the right-hand side whose norm matches the printed iteration-0 residual to 13 digits) was solved with a `petsc4py` harness that replicates `DALinearEqn::createMLRKSP`’s call sequence *in order* — `KSPSetFromOptions` first, hard-coded GMRES/ASM/sub-ILU settings second — under `mpirun -np 4`, the real layout:

```
RESULT reason -9 its 0 finalres 7.091590452305e-04 wall 7.01s
```

Bit-for-bit DAfoam’s failure — same reason, same iteration count, same residual — with `PCView` confirming full parity (ILU fill 1, ordering `rcm`, shift `NONZERO`, pivot-in-blocks, `zeropivot 2.22e-14`).

5.3 Runtime options: reachable, measured, and empty

A standing belief in three independent internal readings held that no PETSc runtime option can reach DAfoam’s ASM sub-block factorization. That was half wrong, and the half matters. PETSc’s `PCSetUp_ASM` calls `KSPSetFromOptions` on each sub-KSP it creates *inside* `KSPSetUp` — after DAfoam’s one visible call site, before DAfoam’s hard-coded per-block overrides. Measured, not argued:

variant (all via <code>PETSC_OPTIONS</code>)	deployed-factor evidence	result
control (parity with DAfoam)	<code>zeropivot 2.22e-14</code> , <code>rcm</code> , shift <code>NONZERO</code>	-9 , iter 0
<code>-sub_pc_factor_nonzeros_along_diagonal</code>	diagonal already has no zeros	-9 , iter 0
<code>-sub_pc_factor_zeropivot 1e-8</code>	“tolerance for zero pivot 1e-08” in <code>PCView</code> — <i>the option reached the factor</i>	-9 , iter 0
<code>-sub_pc_factor_diagonal_fill</code>	applied	-9 , iter 0

Two conclusions, one per column. **Reachability:** factor options do land, so “compiled-in, nothing reachable” was too strong. **Mechanism:** a 10^{-8} zero-pivot threshold with `MAT_SHIFT_NONZERO` active still produces the NaN, so this is *not* a small-pivot-below-threshold event the shift machinery could catch. The failure is factor growth — enormous elements in L/U that overflow when applied — which no shift and no diagonal permutation fixes; only pivoting or completeness does. This also explains, mechanically, the earlier observation that `spilu` reported “exactly singular” at every drop tolerance on the same dumped system while `splu` succeeded at every pivot threshold.

5.4 The one-word change, offline and then in-solver

sub-PC	ordering	reason	iterations	wall	per-block factor size
ILU(1) (DAFoam)	rcm	-9	0	7.0 s	6,963,844 nnz (before NaN)
LU	rcm	2 (converged)	347	68.0 s	100,311,134 nnz (~0.8 GB)
LU	nd	did not finish	—	killed at 600 s	—

Table 9: Offline mechanism test on the dumped system: one change, `PCSetType(subpc, PCLU)` where DAFoam hard-codes PCILU. The `nd` row is an operational finding worth keeping: PETSc’s native LU under nested dissection is pathologically slow on these blocks, so `rcm` is the ordering that works with complete LU here — the exact reverse of its role under ILU, where `rcm` is the ordering that surfaces the NaN.

`PCSetType(MLRsubpc, PCILU)` is hard-coded at `DALinearEqn.C:266-267`, so complete LU needs a recompile. The patch adds an environment-variable switch at that line: when `DAFOAM_SUBPC_TYPE=lu` is set, the local PC type becomes PCLU; otherwise not one line of behaviour changes. In-solver, on B3’s exact `-9` configuration:

```
DAFOAM_SUBPC_TYPE=lu: ASM sub-block PC set to complete LU
Main iteration 0   KSP Residual norm 7.091590452305e-04
Main iteration 400 KSP Residual norm 6.488778512831e-04
Main iteration 500 KSP Residual norm 1.778417281485e-05
Main iteration 667 KSP Residual norm 6.922418564747e-10
**Completed**! Total iterations: 667. PetscConvergedReason: 2. 232.2 s
```

The residual history is worth reading rather than skipping: the first 400 iterations crawl, because the assembled `dRdWTPC` the LU inverts is not the matrix-free operator GMRES applies; then the Krylov space catches the dominant cluster and the residual falls six decades in 267 iterations. Slow-then-superlinear is what a correct strong preconditioner on a mismatched operator looks like. Flat-to-thirteen-digits is what the dead ILU looked like. They are not the same thing.

5.5 The gradient, and its verification

The adjoint linear system depends on the objective, not on the design variable, so the converged solve carries over directly to the field-inversion design variable: 21,000 per-cell β components, same 667 iterations, same reason 2, gradient norm $1.4558046603 \times 10^{-5}$. **This is the first converged field-inversion gradient on a closure-relevant case in this lab.**

cell	h	central FD	adjoint $g[i]$	rel. err
5491 (largest $ g $)	0.05	1.914384790951e-06	1.916018813330e-06	0.085%
5491	0.1	1.907246786675e-06	1.916018813330e-06	0.460%
6740	0.05	1.679642395377e-06	1.678653382471e-06	0.059%
12486	0.05	1.472401682783e-06	1.469472928907e-06	0.199%
6490 (<i>chosen by the verification sweep, never published by the lab</i>)	0.05	1.520027166028e-06	1.519705840939e-06	0.0211%

Table 10: Central-difference verification of the 21,000-component gradient, fresh process per point, cold reset of the processor directories. The unperturbed control run reproduces the adjoint run’s objective to *exactly zero difference* ($1.5279278906359758 \times 10^{-2}$, all 17 digits) — which doubles as a primal-invariance proof across the switch. The doubling of the error from $h = 0.05$ to $h = 0.1$ on cell 5491, with the FD moving *away* from the adjoint value as the step grows, is central-difference truncation behaving exactly as $O(h^2)$ predicts: the agreement is step-limited, not noise-limited.

5.6 Where the remedy comes from, and why it is not exotic

Switching failing ILU sub-blocks to LU is the PETSc developers’ own canonical escalation, not an invention of this lab. From the `petsc-users` archive, verbatim: “*The default preconditioner with ILU(0) on each process is not appropriate for your problem and is producing overflow. Try -sub_pc_type lu and see if that produces a different result*” (Barry Smith, `msg24474`, 2015-03-26); and `msg26973` (Hong, 2015-10-27) demonstrates on a zero-diagonal system that ILU with any shift “does not converge” while `-sub_pc_type lu -sub_pc_factor_shift_type nonzero` converges in 24 iterations. The PETSc FAQ states, verbatim: “*The convergence of many of the preconditioners in PETSc including the default parallel preconditioner block Jacobi depends on the number of processes.*”

The complete upstream DAfoam remedy ladder for a non-converging adjoint, assembled from every place it is stated (the run-script guide, the FAQ, the doxygen DAOPTION page, and the live discussion threads) is: `renumberMesh -overwrite; pcFillLevel 1 → 2; jacMatReOrdering rcm → nd/natural; gmresRestart/gmresMaxIters up; asmOverlap up; upwind schemes`; (since 2025-12) `kahip` decomposition; and `transonicPCOption` for transonic cases. **Nothing in that ladder changes the sub-PC type; the ILU choice itself is never questioned upstream**, and no upstream doc, issue or discussion mentions ILU singularity, zero pivots, or a direct/sub-direct factorization at all — under 8 targeted searches plus 4 doc-page reads. Meanwhile `mdolab/dafoam` discussion #1002, converted to issue #1011 on 2026-07-29 and still open, shows users exhausting the documented ladder on *official tutorials* and stalling: the failure mode this switch was built for is live upstream today.

The shape of a filing. An upstream pull request exposing the sub-PC type as an `adjEqnOption` key — default `ilu`, behaviour unchanged — would be novel, is one option plumb, and is backed by both a measured unblock and named-PETSc-developer precedent. Our local implementation is one hunk at `DALinearEqn.C:266` (`demo-output/website/dafoam/subpclu_patch/DALinearEqn_subpclu.patch`, `1CD44C04`); its default-off behaviour was verified by re-running the unpatched configuration on the patched image and reproducing the `-9` at iteration 0 with all 13 residual digits intact.

5.7 What the change does *not* fix

On the NASA hump (51,626 cells, 51,626 β design variables) the same one-word change moves the failure from catastrophic to honest, and no further. The `-9` is gone — the complete-LU factorization of blocks ILU cannot factor completes, and GMRES iterates with a monotonically descending residual, 1.094138002900 at iteration 0 to 0.9544468674795 at iteration 900 — but it descends slowly, 12.8% in 900 iterations (accelerating: 0.75%, 1.1%, 1.2%, 1.6% per successive hundred) and far from the target. The run was stopped deliberately at roughly iteration 900 when host available memory fell from 30.5 GB to a low-water **1.62 GB** under about 19 GB of LU factors plus a still-filling 2000-vector restart space, on a shared box.

Stated for whoever picks this up: **the hump blocker is no longer the singular factorization — it is Krylov convergence rate against a memory envelope**, which is a different and honestly-named problem.

6 Methods: adversarial verification as practice

Every headline claim in this document has survived an independent adversarial sweep. That phrase is used precisely, and this section says what it buys, because a reader assessing three unfilled defect claims is entitled to know what standard they were held to.

6.1 What a supervisor sweep is

A verification agent is dispatched with the posture that **the claim is wrong until it survives attack**. It does not review the record; it re-derives the record. Concretely, across the four sweeps behind this document:

- **Independent instruments, not re-runs of the lab’s scripts.** The mechanism sweep re-built the processor-to-serial permutation from scratch with its own parsers, from `cellProcAddressing/faceProcAddressing` and the `AdjointIndexing` dumps, then compared against the stored map: identical bit-for-bit in target and sign. The rotation sweep wrote its own NumPy check of the corrected derivative (250 draws, against Richardson central differences of the singularity-free reference) and its own DAfoam driver that wraps `idwarp.USMesh.warpDeriv` itself rather than the lab’s hook.
- **A new validation channel the original never used.** The `AdjointIndexing` dumps carry cell and face centre coordinates; mapping $np=4$ coordinates onto $np=1$ coordinates gives max deviation **0.0** over all 26,149 and 26,227 entries, validating every entry of the permutation exactly and independently of any solved field.
- **Re-measurement from scratch, in the container.** One cell of the phenomenon table ($np=2$, patched) was re-run with the sweep’s own driver, own staging, own container invocation: analytic, FD, absolute error, relative error, baseline C_D and GMRES iteration count *all identical to every printed digit*, and carrying the provenance stamp the archived run lacked. Cost: 86 s wall, 2.9 solver core-min.
- **A point the lab did not choose.** The conditioning sweep verified a finite-difference cell (6490) the lab never published, at a different rank band, and got 0.0211% — tighter than any published cell. A wrong indexing, a stale gradient file, or a lucky triple would not survive that.
- **Cold reproduction of the control, not just the treatment.** The env-off regression was re-run cold by the sweep, on a fresh case copy: reason, iteration count, and all 13 residual digits matched independently of anything the original session wrote.
- **The ledger is audited to the second.** Billed walls were reconciled against log mtimes and internal timings — e.g. five offline runs plus a 600 s kill summing to 693.04 s against 693 billed; a memory trace whose first and last samples bracket 816 s against 819 billed; nine finite-difference runs whose per-run walls sum to *exactly* 642.

6.2 What the sweeps actually found

They did not rubber-stamp. Across the four sweeps the claims survived on every axis — and the sweeps still returned a defect list, which is printed here rather than repaired quietly. The mechanism sweep found five bookkeeping and provenance defects; the phenomenon sweep found two documentation defects and three wording over-reads; the rotation and conditioning sweeps found no defect and two presentation caveats between them. The material ones appear in the body of this document; the consolidated list:

6.3 Pre-registration

Predictions are committed to the repository *before* the runs that test them, and scored exactly as written.

- The cross-residual discriminator was proposed with the prediction “the cross residual will be large.” **HELD**: 3.3×10^2 against a 1.1×10^{-4} floor.

found	disposition
The cross-residual table's numbers appear in no log; the instrument carried a sign error and every arm's log prints a degenerate <code>ratio=2.000000e+00</code>	Stated in full, Section 2.4; values are exact offline corrections and reproduce three ways including a fresh serial run
The either-copy confound bound silently used the assembled matrix — the instrument the same record discredits	Disclosed here; immaterial by a 30× margin even when inflated by the full assembled-vs-AD discrepancy
The cited coloring-crash log no longer exists (the runner truncates per attempt; three attempts shared a tag)	Claim re-established statically in the shipped container, by code rather than by the cited log
The residual-function-identity loophole was left unacknowledged	Closed from the prior record's per-run FD column, and now stated in the body as its own subsection
The case's own ladder record still asserted the superseded verdict, in both directions	Filed as a lesson (L-32); satellites had moved, the primary record had not
A misprinted relative spread (1.7e-06 for 1.7e-05) and an over-broad “any group” invariance sentence (one noise-floor component differs by 11.64%)	Corrected; neither touches a number in any claim
A cost figure of 12.63 core-min that should read 12.80, repeated in a commit message	Corrected in place, with the arithmetic shown
Two headline numbers quoted in different units in a one-line summary; an “improves 34–115x” claim that passes over a fourth row degrading 0.389% to 0.505%	Both carried into this document with the footnote attached
Provenance stamps missing from the headline sweep's logs	Mitigated by a stamped independent re-run of one cell and a stamped pair reproducing the same two numbers

- The coloring-off discriminator was pre-registered with “the gradient will not change, because coloring builds only the preconditioner.” **NOT SCORED** — the discriminator proved unrunnable, so the prediction never met its measurement. It is scored NOT SCORED rather than quietly dropped or retroactively counted.
- The rotation root-cause session committed seven falsification conditions (P1–P7) before any diagnostic ran, including two — “evalMode=exact must *not* help” and “matrix reordering must not move the result at all” — whose failure would have refuted the mechanism. P5 and P6 both held; P7’s designed control came out only partially clean and is reported at its real strength, with the genuinely clean instance found elsewhere.
- The reach matrix (Section 2) registered seven arms with numeric bands and an explicit decision rule for refutation, at 99F5D41D, before any arm ran.

6.4 Cost, stated including the overruns

work	spent (core-min)	budget
Decomposition discriminators (8 container runs)	56.77	45
— of which the five arms that decided the item	12.80	
— of which three attempts at the coloring-off arm, all ending in a measured failure mode rather than a number	43.97	
Mechanism supervisor sweep (one 9 s container run; everything else arithmetic on dumps)	0.30	
Phenomenon supervisor sweep (one np=2 check_totals)	2.90	~30
Conditioning unblock (7 entries: offline ladder, regression control, two converged solves, hump attempt, 9-point FD sweep)	185.0	240
Conditioning supervisor sweep	18.3	22

Table 11: Solver cost. The one overrun — 26% on the discriminators — is confined entirely to the coloring-off arm, and the record volunteers that the second forced attempt should not have been launched after the first showed unbounded memory growth rather than a slow finish. Every run is billed at full cap occupancy for its whole wall clock, including both kills.

6.5 The lessons this arc produced

The method itself is on the record, as numbered lessons in LESSONS.md:

- **L-29.** A guard that replaces a removable singularity with a constant differentiates to a hard zero — at exactly the point every gradient evaluation uses. Audit every such guard on a differentiated path; require the guarded branch to carry the analytic limit, or reparameterise so no guard is needed. And when *testing* derivative code, evaluate at the degenerate/baseline point deliberately and seed with the real objective direction.
- **L-30.** A gate whose threshold lies between two constraint values reports the constraint, not the quantity. Before gating on a reported extremum, find out whether the tool was *told* that number.

- **L-31.** A verification instrument must itself be verified at the component it condemns. When an analytic derivative disagrees with a finite difference, re-measure the finite difference by an independent path — with a neighbouring component as a control — before attributing the gap to the derivative.
- **L-32.** When a verdict moves, the case’s own record must move first; satellites are not the record.
- **L-33.** A correction trained inside a gated regime rebuilds the gate’s own failure at the regime’s floor.
- **L-34.** “Unreachable at runtime” is a claim about the deployed object, not about one call site. Deploy a probe that prints the final object state (for PETSc, PCView on the sub-PC after setup) before declaring a configuration layer unreachable.
- **L-35.** *A converged Krylov solve certifies the operator it was given, not the operator you meant.* When an iterative solve converges cleanly and the downstream answer is still wrong, stop tuning the solve and test the operator: apply the converged solution under an independently evaluated instance of the operator — serial where the suspect is parallel, assembled where the suspect is matrix-free — and compare the residual to that instance’s own floor. It is a mat-vec, not a solve: seconds, not core-minutes. Two footnotes: map vectors across decompositions with the integer addressing files (sign on flipped faces), never coordinate matching, and validate the map on the primal state first; and diff operator *actions* on mapped vectors, never operator *entries*.

A Reproduction appendix

A.1 Environments

image / package	identity
dafoam/opt-packages:latest IDWarp 2.6.2 (inside the above)	DAFoam 5.0.0, OpenFOAM v2506, PETSc 3.15.5. The stock toolchain; all phenomenon and mechanism measurements digest sha256:9d45679d55fd47f5ca7afd99cabb86c7c2729cf2acf34c438eb33af5290f07fc; installed idwarp/*.py md5-identical to tag v2.6.2, tag commit 647FD8FC
patched IDWarp	scratch clone at /home/ubuntu/certonomous-runs/W5-patch/idwarp; git diff byte-identical to rotation_branch/idwarp_v2.6.2_degenerate_branch_fix.patch (2 files, +44 lines). Mounted, never installed in place
dafoam-subpclu:v1	local-only docker commit of the stock image with one hunk at DALinearEqn.C:266 rebuilt in all three AD modes. Nothing pushed

A.2 Switches and stamps

```
# Which IDWarp is actually loaded -- stamped into every log by the driver.
IDWARP_IMPORTED_FROM: /patch/idwarp/idwarp/__init__.py      # patched arm
IDWARP_IMPORTED_FROM: <container site-packages>              # stock arm

# Sub-block preconditioner switch (patched image only; default off).
docker run -e DAFOAM_SUBPC_TYPE=lu ... dafoam-subpclu:v1
# Log line emitted on block 0 when active:
#   DAFOAM_SUBPC_TYPE=lu: ASM sub-block PC set to complete LU
```

```
# The IDWarp workaround, for anyone who needs a consistent gradient today
# (mesh-quality price NOT measured -- see Section 4.7):
meshOptions = {..., "useRotations": False, ...}
```

A.3 Varying the decomposition without falling into the trap

Editing `system/decomposeParDict` directly is silently ineffective: `pyDAFoam` regenerates it. Set the `daOption` instead, and *read the decomposition back out of the log*:

```
daOptions = {
  # ...
  "decomposeParDict": {"method": "simple", "simpleCoeffs": {"n": [4, 1, 1]}},
  # default when unset is {"method": "scotch"} -- pyDAFoam.py:590-591
}
```

```
# Verify the edit took effect -- this line is the check that catches the trap:
grep "Decomposition method" run.log
#   Decomposition method scotch [4] (region region0)
#   Decomposition method simple [4] (region region0)
```

A.4 The cross-residual protocol

Pure `DAFoam` public API; no rebuild required. This is the instrument any candidate fix must satisfy.

```
# 1. Per decomposition: run the primal + adjoint, then dump
#   psi   = DAFoamSolver.psi
#   state = the converged primal state vector
#   b     = dFdW (the adjoint RHS)
#   with, in daOptions:
#     "writeJacobians": ["adjointIndexing"]
#   which emits AdjointIndexing_<rank>_of_<n>.txt (indices + cell/face centres).

# 2. Build the exact integer permutation to serial ordering from
#   processor*/constant/polyMesh/cellProcAddressing
#   processor*/constant/polyMesh/faceProcAddressing (sign => flipped face)
#   NEVER by coordinate matching. Validate it on the PRIMAL state first:
#   duplicated processor-face phi copies must agree to machine precision
#   (measured here: 2.7e-15). The AdjointIndexing centres give a second,
#   exact channel: max |dxyz| must be 0.0.

# 3. Evaluate the true residual under the SERIAL operator, matrix-free:
#   r = solverAD.calcJacTVecProduct(state -> residual) applied to mapped psi
#   SIGN CONVENTION: OpenMDAO seeds -dF/dW, so the system is A^T psi = -b.
#   An instrument computing (A^T psi - b) prints a degenerate ratio of exactly
#   2.0 for its own-operator control -- that is the tell that the sign is wrong.
#   Correct offline as res + 2b, or fix the instrument.

# 4. Judge against the SERIAL operator's own floor, not against zero:
#   np=1 control ..... 1.14e-04 of ||b|| <- the floor
#   np=4 simple 4x1x1 .... 9.37e-02
#   np=4 scotch ..... 3.288e+02 <- the defect
```

Lab scripts implementing this: `W4-a4-discriminators/runScript_w4.py` (tasks `w4_dump`, `w4_crossres`, `w4_crossres2`), `build_maps.py`, `analyze_mats.py`.

A.5 Reproducing the IDWarp defect in isolation

No DAfoam, no OpenFOAM, no pyGeo, no CFD — IDWarp’s own test mesh, options, displacement and verification routine:

```
# input_files/symm_block.cgns, md5 0d5a0411cbaa00f6afd88a9430a7f634,
# from upstream's own input_files/get-input-files.sh
mesh.verifyWarpDeriv(dXv_warp, solverVec=False, dofStart=0, dofEnd=5)
# Rotations ON : DOF 0 AD 2.87055669475 vs FD -115.875789153 -> 210.16%
# Rotations OFF: DOF 0 AD 2.87055669475 vs FD 2.87055639160 -> -1.06e-05%
# ~ the AD column is BIT-IDENTICAL between the two runs: the proof that
# the rotation term was never in the derivative.
# Adversarial check on the FD: stable to 6 significant figures over
# h = 1e-4, 1e-5, 1e-6; roundoff only at 1e-8. The FD is converged.
```

A.6 Rebuilding the sub-LU image

```
docker run -d --name dafoam-build dafoam/opt-packages:latest sleep infinity
# patch src/adjoint/DALinearEqn/DALinearEqn.C at line 266 (one hunk, env-gated)
# then, INSIDE the container, for each of the three AD modes:
source /home/dafoamuser/dafoam/loadDAfoam.sh # original
cd repos/dafoam && wmakeLnInclude src/adjoint && (cd src/adjoint && wmake -j 8)
# ADR and ADF: sed WM_AD_MODE in OpenFOAM-AD/etc/bashrc, re-source, wmake again
docker commit dafoam-build dafoam-subclu:v1
# Caution: 'set -e' before sourcing the OpenFOAM environment kills the shell.
# Source first. All three libDASolver*.so relink in seconds.
```

A.7 Artifact index

All paths are in the Certonomous repository unless marked as a run directory. Run directories live outside the repository, on the measurement host.

artifact	what it holds
Repository: demo-output/website/dafoam/	
VERIFICATION_A4_decomposition_supervisor_sweep.md (27D25762)	The verified phenomenon table, five verification axes, the hanging-node refutation
DISCRIMINATORS_A4_decomposition_mechanism.md (+ 2026-08-04 amendments)	The four discriminators, the cross-residual, the localization, the coloring-off finding
VERIFICATION_A4_mechanism_supervisor_sweep.md (8E0A08BC)	Adversarial sweep of the mechanism: independent map, independent scripts, fresh serial run, five defects found
UPSTREAM_BUG_REPORT_decomposition_adjoint.md (CD573DF3)	The prepared, unfilled report (D1), cited here at its committed state. A further amendment from the papers-protocol work — carrying the two-claims correction of Section 3 into the report’s own header, plus the 2018 paper’s own dCD/du0 check run at two decompositions — was in progress while this document was written and may have landed since
UPSTREAM_BUG_REPORT_mesh_warpDeriv.md	The prepared, unfilled report (D2)
DAFOAM_PAPERS_VERIFICATION_PROTOCOLS.md (BF6AC53B)	Three method papers read in full; the protocol table of Section 3

artifact	what it holds
LIAISON_NOVELTY_SWEEP_decomposition_defect.md (3C74DC03)	63 searches, 10 venues, the query log, the near-misses, the PETSc precedent
ROOTCAUSE_getRotationMatrix3d.md, PATCH_getRotationMatrix3d.md	The line, the mechanism, the derivation-before-code, the acceptance results
VERIFICATION_rotation_patch_supervisor_sweep.md (6782D33A)	Adversarial sweep of D2: math re-derived, own driver, own reproduction
ladder-b/W4_ADJOINT_PC_UNBLOCK.md	The conditioning wall, the offline ladder, the sub-LU unblock, the FD gate
VERIFICATION_cbfs_unblock_supervisor_sweep.md (9C19CCC8)	Adversarial sweep of D3: image-vs-diff audit, recomputed FD table, new cell, cold regression
DEFECT_REACH_decomposition_cases.md (99F5D41D)	The pre-registered reach matrix. Results in progress at the time of writing
subpclu_patch/DALinearEqn_subpclu.patch (1CD44C04)	The one-hunk sub-PC source diff
rotation_branch/idwarp_v2.6.2_degenerate_branch_fix.patch	The four-line derivative correction (2 files, +44 lines)
PROOF.md §25.5, §23–24, §25.2–25.3	The primary measurement record
LESSONS.md L-29 – L-35	The method-of-investigation thread
Run directories: /home/ubuntu/certonomous-runs/	
W4-a4-stepsweep/	The phenomenon table’s per-cell logs
W4-a4-discriminators/	Per-arm dirs, dumped operators and vectors, ledger.txt, the analysis scripts
W4-a4-verify-audit/	The mechanism sweep’s own staged case, own mapped vectors, audit_crossres.log
VERIFY-a4-np2-20260804/	The independent re-run of one phenomenon cell
W4-defect-reach/	The reach matrix’s arms and ledger.txt
W5-idwarp-source/, W5-patch/, W5-rotation-branch/	IDWarp source clone, patched clone, and the A5 case copies
W4-adjoint-pc-unblock/	Offline petsc4py ladder, CBFS and hump runs, the 9-point FD sweep
W4-verify-sweep/	The conditioning sweep’s isolated case copy and its own logs

A.8 What is still open

1. **D1, mechanism to a line.** The subsystem is named and the spatial signature is pinned; the defective source line is not. Reaching it requires an instrumented rebuild of `libDASolver.so`.
2. **D1, reproducer on a stock upstream tutorial.** The measured case is lab-built — its mesh recipe and case directories are archived and shippable, but it is not an official tutorial. A tutorial-based reproducer varying the `decomposeParDict daOption` on an official case should exist before filing.
3. **D1, reach.** The pre-registered breadth matrix has run but is not yet scored.
4. **D2, the primary reference.** The IDWarp AIAA paper is unread and unobtainable from this host; the “undocumented” characterisation rests on code, docs and release notes only, and is stated conditionally throughout.
5. **D2, the second regime.** Real, measured at $\sim 1.26\%$ on ONERA M6, untouched by the shipped patch by design. The reparameterisation that subsumes both regimes is derived and numerically validated but not implemented, and cannot be primal-bit-identical.

6. **D2, the workaround's price.** The mesh-quality cost of `useRotations`: `False` under large deformation has not been measured.
7. **D3, the hump.** Krylov convergence rate against a memory envelope. A prepared configuration wrapping the exact sub-solve in Richardson with a capped restart is staged but unrun.

Filing status, restated. Nothing in this document has been filed, sent, or posted. Whether any of it is submitted upstream is the lab principal's decision, not the authors'. The two prepared reports and the comment-on-#57 remain drafts.
--